AI-Assisted Development and Git Mastery Objective

Made by Javier Gallego Narváez

MEDAC

Instituto Oficial de Formación Profesional

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git init
Initialized empty Git repository in C:/Users/galle/OneDrive/Escritorio/Tarea20/.git/
```

This command is used to initialize a local repository. It creates a hidden folder named `.git` inside my project directory, which allows Git to start tracking changes, managing versions, and recording the history of my files from that moment forward.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git config --global user.name "Javier"
```

Then, this command sets the identity for all the Git operations on the computer. By providing my name, Git can label every "commit" mades, allowing collaborators to see who authored specific changes in the project history.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git config --global user.email "jgn0018@alu.medac.es"
```

This command links my email address to my Git activity. It ensures that every contribution I save is tied to my professional identity, which is essential for platforms like GitHub to recognize the work and display my profile picture next to my commits.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git add tarea.html
```

This command moves the file to the Staging Area. It tells Git that I want to include the specific changes made to `tarea.html` in my next commit, acting as a preparation step before permanently recording the history.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   tarea.html
```

Then, I used this command to check the current state of my project. It shows me which branch I am on and confirms that `tarea.html` is staged and ready to be committed.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git commit -m "Initial commit: Functional unit converter created with AI"
[main (root-commit) 9fed87f] Initial commit: Functional unit converter created with AI
 1 file changed, 82 insertions(+)
 create mode 100644 tarea.html
```

With this command, I saved the first official version of my project in the local repository. The `-m` flag allowed me to add a descriptive message, creating a permanent snapshot of my code at this specific point in time.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git branch feature-update
```

I used this command to create a new branch called `feature-update`. This allowed me to work on a new feature in an isolated environment without affecting the stable code in the main branch.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git checkout feature-update
Switched to branch 'feature-update'
```

After creating the new branch, I ran this command to switch from the main branch to `feature-update`. This ensured that any new code I wrote or modified from that moment on would only be saved within this specific branch.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git add tarea.html
```

Then, I used this command to stage my modified file. It tells Git that I want to include the new changes I made to `tarea.html` in my next commit, moving the file from the working directory to the staging area.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git commit -m "Visual update: Button color changed"
[feature-update 57e96de] Visual update: Button color changed
 1 file changed, 9 insertions(+), 9 deletions(-)
```

After modifying the code in my new branch, I used this command to save those specific changes. This created a new commit within the `feature-update` branch, documenting the visual improvements I made to the button before merging them back to the main project.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git checkout main
Switched to branch 'main'
```

Once I finished the updates in the secondary branch, I used this command to switch back to the **main** branch. This is a necessary step before merging my new changes into the master version of the project.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git merge feature-update
Updating 9fed87f..57e96de
Fast-forward
 tarea.html | 18 +++++++++---------
 1 file changed, 9 insertions(+), 9 deletions(-)
```

I used this command to combine the changes from my `feature-update` branch into the **main** branch. As seen in the terminal, Git performed a "Fast-forward" merge, successfully integrating the new code into the project's primary history.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git branch -M main
```

I used this command to rename my current primary branch to main. This is a standard step to ensure the branch name matches the default naming convention used by GitHub, making the deployment process smoother.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git remote add origin https://github.com/jgn0018-droid/conversor.git
```

Then, I used this command to connect my local repository on my computer to the remote repository I created on GitHub. By adding the "origin" alias, I established a link that allows me to upload my code and project history to the cloud.

```
PS C:\Users\galle\OneDrive\Escritorio\Tarea20> git push -u origin main
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (6/6), 1.90 KiB | 1.90 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), done.
To https://github.com/jgn0018-droid/conversor.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

This was the final step to upload my project to the cloud. I used this command to push my local commits and branch history to the GitHub repository. The `-u` flag linked my local main branch to the remote one, making it easy to sync my work in the future.

MEDAC

Instituto Oficial de Formación Profesional